

Estadística descriptiva con Python

Unidad V: análisis de datos con pandas, NumPy y Matplotlib

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 12, Sesión 1 (Sesión 23)

Contexto

Dataset base

Medidas descriptivas

Datos faltantes

Agrupación

Distribuciones y gráficos

Cierre

Contexto

- En las sesiones anteriores se trabajó el flujo básico con **pandas**: crear, leer, inspeccionar, filtrar, limpiar y agrupar datos.
- Esta sesión usa ese flujo para obtener resúmenes numéricos e interpretar la variabilidad de un conjunto de datos.
- El foco está en estadística descriptiva: describir una muestra antes de construir modelos más complejos.

Meta de la sesión

Calcular e interpretar medidas de tendencia central, dispersión y distribución usando **pandas**, **NumPy** y **Matplotlib**.

- **pandas**: organización tabular, selección de columnas, resúmenes y agrupaciones.
- **NumPy**: cálculos numéricos y generación de datos reproducibles.
- **Matplotlib**: visualización de distribuciones y relaciones entre variables.

Flujo de análisis

Leer datos → inspeccionar → limpiar → resumir → visualizar → interpretar.

Al terminar la sesión, deberían poder:

- calcular media, mediana, varianza y desviación estándar;
- usar `describe()` y `quantile()` para resumir una columna;
- decidir cuándo limpiar valores faltantes antes de calcular estadísticas;
- comparar grupos con `groupby` y `agg`;
- interpretar histogramas, dispersión y valores extremos;
- comunicar conclusiones breves a partir de tablas y gráficos.

Dataset base

Dataset base

Catálogo de observaciones

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 df = pd.DataFrame({
6     "objeto": ["A", "B", "C", "D", "E", "F", "G", "H"],
7     "tipo": ["estrella", "estrella", "planeta", "estrella",
8             "planeta", "galaxia", "galaxia", "estrella"],
9     "masa": [1.0, 1.4, 0.003, 0.8, 0.002, 1.2e11, 8.5e10, 2.1],
10    "temperatura": [5800, 7200, 290, 4300, 160, np.nan, np.nan, 9500],
11    "distancia_pc": [4.2, 12.5, 0.00001, 8.1, 0.00002, 1.5e6, 2.1e6, 25.0],
12    "magnitud": [4.8, 2.1, np.nan, 7.3, np.nan, 12.2, 13.1, 0.8]
13 })
```



```
1 df.head()  
2 df.info()  
3 df.describe()  
4 df.isna().sum()
```

- `info()` permite revisar tipos de datos y valores no nulos.
- `describe()` entrega un resumen inicial de columnas numéricas.
- `isna().sum()` muestra cuántos valores faltan por columna.

Medidas descriptivas

Medidas descriptivas

Tendencia central

Media

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Mediana

Valor central al ordenar los datos. Es menos sensible a valores extremos que la media.

- La media resume el centro aritmético.
- La mediana resume el centro por posición.
- Si hay valores extremos, ambas pueden diferir bastante.

Medidas descriptivas

Media y mediana en pandas

```
1 temperatura_media = df["temperatura"].mean()
2 temperatura_mediana = df["temperatura"].median()
3
4 print(temperatura_media)
5 print(temperatura_mediana)
```

- Por defecto, pandas ignora **NaN** en muchos cálculos estadísticos.
- Aun así, los valores faltantes deben reportarse antes de interpretar resultados.

Medidas descriptivas

Dispersión

Varianza

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Desviación estándar

$$s = \sqrt{s^2}$$

- La desviación estándar mide dispersión en las mismas unidades de la variable.
- Valores grandes indican datos más extendidos respecto del promedio.

Medidas descriptivas

Dispersión en pandas

```
1 resumen_temp = pd.Series({
2     "media": df["temperatura"].mean(),
3     "mediana": df["temperatura"].median(),
4     "varianza": df["temperatura"].var(),
5     "desviacion": df["temperatura"].std(),
6     "minimo": df["temperatura"].min(),
7     "maximo": df["temperatura"].max()
8 })
9
10 print(resumen_temp)
```

Medidas descriptivas

Cuantiles

```
1 df["temperatura"].quantile([0.25, 0.50, 0.75])
```

- El cuantil 0.25 marca el primer cuartil.
- El cuantil 0.50 coincide con la mediana.
- El cuantil 0.75 marca el tercer cuartil.

Uso

Los cuantiles ayudan a describir la distribución sin depender solo de la media.

Datos faltantes

```
1 numericas = df.select_dtypes(include=[np.number])
2
3 print(numericas.columns)
4 print(numericas.describe())
```

- Esta selección evita aplicar estadísticas a columnas categóricas.
- Es útil cuando el DataFrame tiene texto, categorías y números mezclados.

Datos faltantes

Eliminar o rellenar

```
1 df_sin_nan = df.dropna(subset=["temperatura"])
2
3 media_temp = df["temperatura"].mean()
4 df_relleno = df.copy()
5 df_relleno["temperatura"] = (
6     df_relleno["temperatura"].fillna(media_temp)
7 )
```

Criterio

Rellenar valores faltantes puede ser práctico, pero no siempre es físicamente justificable. La decisión debe explicarse.

Agrupación

Agrupación

Resumen por categoría

```
1 df.groupby("tipo")["distancia_pc"].mean()
```

- **groupby** separa el DataFrame según una categoría.
- Luego se aplica una operación a cada grupo.

Agrupación

Varias estadísticas con **agg**

```
1 resumen_por_tipo = df.groupby("tipo").agg({
2     "distancia_pc": ["count", "mean", "median", "std"],
3     "temperatura": ["mean", "median"],
4     "magnitud": ["mean", "min"]
5 })
6
7 print(resumen_por_tipo)
```

Lectura

Esta tabla permite comparar grupos y detectar si una categoría tiene pocas observaciones.

Distribuciones y gráficos

Distribuciones y gráficos

Histograma

- Un histograma muestra cómo se distribuyen los valores de una variable.
- El eje horizontal representa rangos de valores.
- El eje vertical representa frecuencia o conteo.
- El número de **bins** controla la partición del rango.

Interpretación

Revisar forma, concentración, dispersión y posibles valores extremos.

Distribuciones y gráficos

Histograma con pandas

```
1 df_sin_nan["temperatura"].hist(bins=5, edgecolor="black")
2
3 plt.xlabel("Temperatura [K]")
4 plt.ylabel("Frecuencia")
5 plt.title("Distribución de temperaturas")
6 plt.grid(alpha=0.3)
7 plt.show()
```


Distribuciones y gráficos

Scatter para comparar variables

```
1 estrellas = df[df["tipo"] == "estrella"]
2
3 plt.scatter(estrellas["temperatura"], estrellas["magnitud"])
4 plt.xlabel("Temperatura [K]")
5 plt.ylabel("Magnitud")
6 plt.title("Temperatura vs magnitud")
7 plt.grid(alpha=0.3)
8 plt.show()
```

- Un scatter ayuda a explorar relaciones entre dos variables.
- Antes de concluir, conviene revisar escala, unidades y tamaño de muestra.

Distribuciones y gráficos

Boxplot por grupo

```
1 df_sin_nan.boxplot(column="temperatura", by="tipo")
2
3 plt.suptitle("")
4 plt.title("Temperatura por tipo de objeto")
5 plt.xlabel("Tipo")
6 plt.ylabel("Temperatura [K]")
7 plt.show()
```

- El boxplot resume mediana, cuartiles y valores extremos.
- Es útil para comparar distribuciones entre categorías.

Cierre

1. Leer o crear un DataFrame.
2. Revisar columnas, tipos y valores faltantes.
3. Seleccionar columnas numéricas.
4. Calcular **describe()**, media, mediana y desviación estándar.
5. Agrupar por una categoría relevante.
6. Graficar una distribución y una relación entre variables.
7. Escribir tres conclusiones breves.

- La estadística descriptiva resume datos antes de modelar.
- La media, mediana y desviación estándar deben interpretarse junto con valores faltantes y valores extremos.
- **groupby** permite comparar subconjuntos de datos.
- Histogramas, scatter y boxplots ayudan a leer patrones que no siempre aparecen en una tabla.